

SLM OS — Technical Build Plan

v1 — Phase 1: Fraud Alert Triage Vertical

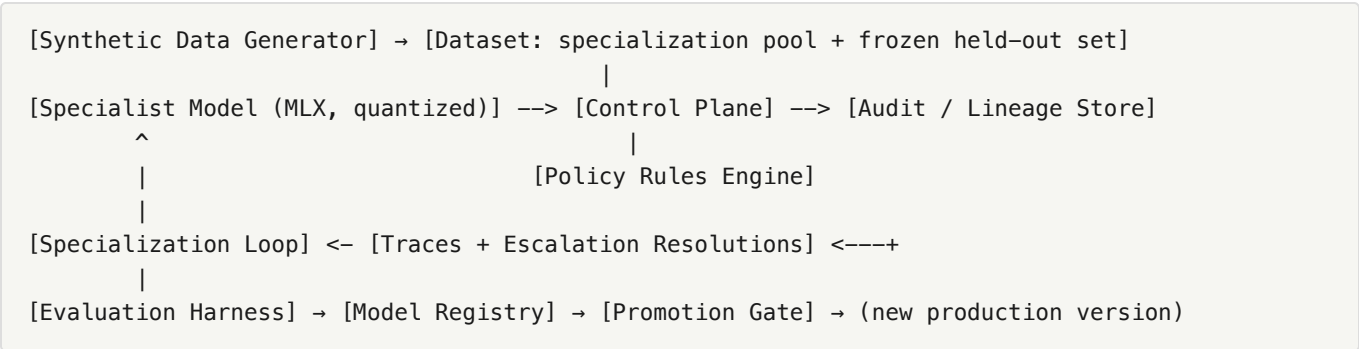
August 2026

This is a design and planning document only. Nothing described here has been implemented yet.

Guiding Constraint

Everything runs locally first, on a MacBook Air, with a Mac Mini arriving in roughly 45 days for heavier iteration. That rules out anything requiring a GPU cluster, external hosted training, or heavy MLOps infrastructure. The stack is Python end-to-end, minimal dependencies, built so pieces can be swapped for cloud-scale equivalents later without a redesign.

Component Architecture



Technology Choices

Layer	Choice	Rationale
Model runtime	MLX / mlx-lm	Native to Apple Silicon; supports both quantized inference and LoRA fine-tuning on-device — one framework covers inference and specialization without a cloud GPU
Model size	3B–7B quantized (e.g. Qwen2.5-3B/7B-Instruct or Llama-3.2-3B, 4-bit)	Fits MacBook Air memory; fast enough for iterative development
Control plane	Python + FastAPI	Simple recommend-endpoint now; can be extracted into a standalone service in Phase 2 without a rewrite
Output contract	JSON Schema + constrained/grammar decoding (e.g. out lines, or MLX grammar-constrained sampling)	The model is prevented from emitting an out-of-schema token in the first place, rather than relying on catching bad output after the fact. Any output that still fails

		validation is a hard failure — automatically escalated and logged
Policy rules	Declarative YAML rule set, evaluated by a small rule function	Avoids over-engineering with a rules engine like OPA/Rego at this stage; kept swappable behind a stable interface
Audit / lineage store	SQLite, append-only tables, each row hash-chained to the previous row	File-based, zero operational overhead; hash-chaining gives tamper-evidence sufficient for an MRM/audit conversation without needing a real blockchain
Synthetic data generation	A larger model (API-based) generates diverse fraud-alert scenarios and ground-truth recommendations; a human-reviewed sample checks quality	Diversity and correctness must be verified before anything is trained on it — data quality is the single largest risk to the whole loop
Fine-tuning	mlx-lm LoRA, small rank, few epochs, adapters fused after training	Matches the "light QLoRA" approach in the roadmap; fusing keeps production inference simple — one quantized checkpoint, not adapter-swapping at serve time
Model registry	JSON manifest per version — parent version id, training-data hash, evaluation results, promotion status — stored alongside SQLite	Gives real lineage without adopting MLflow/W&B-class infrastructure this early
Orchestration	Plain Python CLI with subcommands (generate-data, run-baseline, specialize, evaluate, promote)	No workflow engine (Airflow/Prefect) at this scale — a CLI is auditable, debuggable, and matches the "narrow, not broad" build philosophy

Recommendation Schema (illustrative shape)

```
{
  "alert_id": "string",
  "action": "approve | decline | escalate_l2 | request_more_info",
  "risk_score": 0.0,
  "confidence": 0.0,
  "policy_flags": ["string"],
  "rationale": "string, length-bounded"
}
```

The control plane validates the schema, applies policy rules (e.g. certain transaction types always escalate regardless of model output; confidence below threshold forces escalation), makes the final decision, and writes the full trace — input, raw model output, validation result, policy checks applied, final decision, model version id — to the audit log.

Specialization Loop

1. Pull traces since the last cycle from the audit store.

2. Filter to the traces that matter: low-confidence cases, cases where the policy engine overrode the model, and cases escalated to the larger model or a human.
3. Convert filtered traces into synthetic training examples that preserve the structural pattern of what made the case hard, without carrying the literal record — this is the privacy-safe conversion step, built for real now so it transfers directly to a deployment with genuinely sensitive data later.
4. Merge into the specialization pool and run a LoRA fine-tune; fuse adapters into a candidate model version.
5. Score the candidate against the frozen held-out set (never used in training) and the regression / challenge set (curated hard cases that must never break).
6. Apply the promotion gate (below). Pass → promote and swap the production pointer, logging the promotion event. Fail → archive the candidate with its evaluation report; production version is unchanged.

Distillation Approach

Two distinct mechanisms are in use, and one is deliberately excluded:

In use — response-level (behavioral) distillation

A larger model's outputs become training signal for the small specialist model in two ways:

- **Initial data generation:** the larger model produces the first synthetic scenario/label pairs used to establish the baseline specialist.
- **Continuous, via the escalation path:** every case the small model escalates (low confidence, policy override, or genuinely hard case) and that gets resolved by the larger model becomes a teacher demonstration. These resolutions feed directly into the next specialization cycle, making distillation continuous rather than a one-time step, and reusing the escalation mechanism that already exists for cost control.

In use — rationale distillation

The rationale field in the recommendation schema is trained to reflect the teacher's justification style, not just its final action — this improves interpretability and audit quality, not only accuracy.

Deliberately excluded — logit-level (soft-label) knowledge distillation

Classic KD trains the student to match the teacher's output probability distribution, not just its final answer. This is not used here because:

- An API-based teacher model does not expose token-level log-probabilities to distill against.
- Even with access, the teacher and student are unlikely to share a tokenizer or architecture family, making logit-matching meaningless without extra alignment work.
- Logit-level KD only becomes practical if the teacher is a larger open-weights model in the same family, run locally — more compute than Phase 1 warrants.

Net effect: the loop uses demonstration-level distillation (what the teacher decided and why), not distribution-level distillation (the exact shape of the teacher's confidence).

Evaluation & Promotion Gate

Gate	Criterion
Accuracy	No regression against the frozen held-out set relative to the current production baseline
Policy / safety compliance	Zero tolerance on recommendations that violate defined policy constraints
Confidence calibration	Confidence scores continue to correlate with correctness, so escalation routing stays reliable
Regression / challenge set	No new failures on the curated set of edge cases and prior known failure patterns

A candidate must clear every gate to be promoted. Every evaluation run, pass/fail result, and promotion decision is itself logged and replayable.

Phase 1 Deliverable (5 weeks, MacBook Air)

A CLI-driven, end-to-end reproducible run producing: baseline accuracy → one specialization cycle (including at least one escalation-driven distillation pass) → post-cycle accuracy → full audit trail export. This is what turns the product brief's placeholder metrics section into a real, measured before/after number.

Known Technical Risks

- **Structured output reliability.** Small models are unreliable at strict structured output without constrained decoding — this piece is load-bearing for the "models only recommend" guarantee and should not be skipped or deferred.
- **Synthetic data quality.** The single biggest failure mode for both training and evaluation. Budget real time for human review sampling, not just generation volume.
- **mlx-lm LoRA maturity.** Younger and less battle-tested than the CUDA/PEFT ecosystem. Worth a short spike early to confirm the fine-tuning and fusing workflow behaves as expected before committing the full loop to it.
- **Escalation-driven distillation volume.** In early cycles, escalation volume may be too low or too narrow to meaningfully improve the model — the loop should tolerate a "not enough new signal yet" outcome without forcing a promotion.

How This Maps to the Roadmap

Roadmap phase	This document covers
Phase 1 — Vertical reference implementation	Everything above: schema, control plane, baseline model, specialization loop, distillation approach, evaluation harness
Phase 2 — Harden into platform core	Extract control plane and specialization engine into reusable, versioned components; move audit store to Postgres if needed; add a second domain pack to prove reusability

Phase 3 — Platform shape & credibility	Open-source the core and reference implementation; publish architecture decision records
Phase 4 — Enterprise packaging	Defined separately once Phase 1 results exist

Status: design document only. No implementation has started.